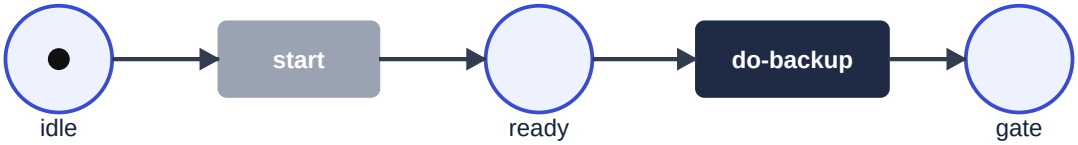
A FIRST-PRINCIPLES GUIDE TO THE petri-flow REPOSITORY

PetriFlow, explained from zero

What problem it solves, how every decision was made, how the code is built, what the proofs really mean, where the holes are, and what still needs research. Plain English. Many pictures.

Written 8 October 2026 from a direct read of the source at C:\petri-flow\petri-flow



WHAT IS INSIDE

Part A The problem, and Petri nets from scratch (chapters 1 to 3)
Part B The architecture: layers, packages, gate, rules, adapters, engine (chapters 4 to 9)
Part C The honest part: what is proven, what is not, loopholes, open research (chapters 10 to 12)
Part D A reading path through the code, a glossary, and a self-test (chapter 13)

Reading level: no prior knowledge of Petri nets, formal methods, or this codebase is assumed.
Every claim about behaviour points at a file so you can check it yourself.

Contents

Contents 2

How to read this guide 4

1. The problem, from first principles 5

1.1 A robot helper with sharp tools 5

1.2 Four ways people tried to control it, and why each one leaks 5

1.3 Where should the rule live? 5

1.4 The second problem: can you trust the gate? 6

2. Petri nets for a complete beginner 7

2.1 Cups, marbles, and machines 7

2.2 Doing two things at once: the coffee net 7

2.3 Every possible future: reachable states 7

2.4 Choice and budget 8

2.5 Why not a state machine or a DAG? 9

3. The big trick: a rule becomes a shape 10

3.1 require A before B 10

3.2 require human-approval before B 10

3.3 block A 11

3.4 limit A to N per session / limit A to N per action 11

3.5 map tool.field pattern as name 12

3.6 Rules compose by intersection 12

3.7 What 'verified at compile time' means for these rules 13

4. The map of the repository 14

4.1 Two products in one repo 14

4.2 The layer diagram 14

4.3 Folder by folder 14

4.4 Tooling choices 15

5. How it was made: the story behind the decisions 16

5.1 Before the repo: petri-ts 16

5.2 0.1.0: the workflow engine, then the gate 16

5.3 0.2.0: the gate must be stateless 17

5.4 0.3.0: block messages the model can act on 17

5.5 0.3.1: replay from history 17

5.6 0.4.0: 'failed' is not always an exception 17

6. Inside the gate: one tool call's journey 18

6.1 Phase 1: every net gives a verdict 18

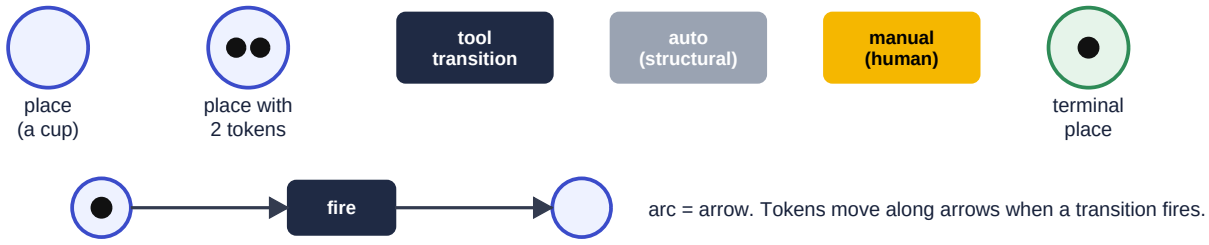
6.2 Phase 2: human approvals . . . 18
6.3 Phase 3: semantic validation with rollback . . . 19
6.4 Phase 4: commit, with a second look . . . 19
6.5 Deferred transitions in detail . . . 19
6.6 autoAdvance: the net moves itself when it can . . . 19
6.7 Tool mapping . . . 19
6.8 The manager and its state . . . 20
7. Inside the rules compiler . . . 21
8. Adapters: plugging into real agent frameworks . . . 22
8.1 Vercel AI SDK: the reference adapter . . . 22
8.2 Claude Code: a process that forgets everything . . . 23
8.3 OpenClaw: inventing ids . . . 23
8.4 pi-assistant: the smart hand-written nets . . . 23
9. The other half: the workflow engine . . . 24
9.1 Defining a workflow . . . 24
9.2 Running: executor and scheduler . . . 24
9.3 Persistence . . . 24
9.4 Analysis and the CLI . . . 25
9.5 Two example nets worth studying . . . 25
10. What 'proven' really means here . . . 26
11. What it is not telling you: loopholes . . . 27
Group 1: the gate sees names, not reality . . . 27
Group 2: state can be wrong or lost . . . 27
Group 3: quiet behaviours inside the gate . . . 28
Group 4: the engine side . . . 28
Group 5: documentation drift found while reading . . . 28
12. What needs research . . . 30
13. Walk the code yourself . . . 32
13.1 A reading order . . . 32
13.2 Commands to try . . . 32
13.3 Glossary . . . 32
13.4 Self-test . . . 33

How to read this guide

This guide teaches one open source repository called **petri-flow** (the product is called **PetriFlow**). It was written by Josh Tuddenham and lives on GitHub under joshuaisaact/petri-flow. You do not need to know anything before you start. Each chapter builds on the one before.

Three rules were used when writing it:

- **Short sentences, everyday words.** If a technical word is needed, it is explained the first time it appears.
- **A picture for every idea.** All diagrams use the same visual language, shown below.
- **Every claim points at a file.** If this guide says 'the gate does X', the file and function are named so you can verify it.



The visual language used in every diagram in this guide.

> **IN SIMPLE WORDS** **The one-sentence summary of the whole repo**
>
> PetriFlow turns a safety rule like 'back up before you delete' into a tiny board game with marbles and cups. The AI agent is only allowed to use a tool if the marbles are in the right cups. Because the game is small, a computer can check every possible move in advance and promise the rule can never be broken.

Boxes like the one above appear through the guide. Green boxes restate an idea in the simplest words. Blue boxes explain why the author chose to build something a certain way. Red boxes point at loopholes. Purple boxes list open research questions.

Version snapshot read for this guide: engine 0.3.0, gate 0.3.2, rules 0.3.0, vercel-ai 0.4.0, cli 0.1.0. The folder had no git history, so the guide relies on CHANGELOG.md for the timeline.

1. The problem, from first principles

1.1 A robot helper with sharp tools

Imagine you hire a helper. The helper is clever but a little unpredictable. You give it tools: a shredder (delete files), a megaphone (send messages to a Slack channel), a crane (deploy code to production), and a photocopier (make a backup). You tell it: *always photocopy a document before you shred it*.

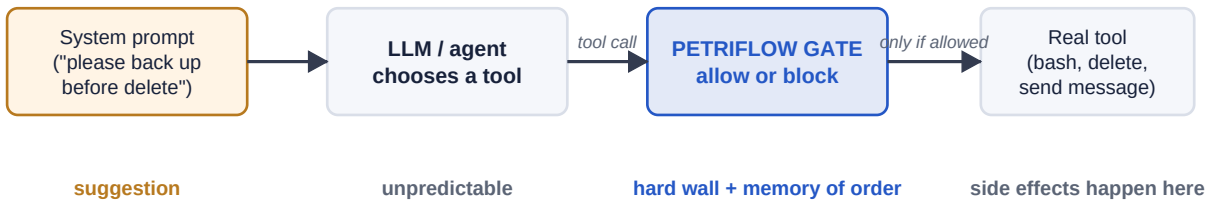
An AI agent is exactly this helper. A large language model (LLM) reads a task, decides which tool to call, calls it, reads the result, and decides again. The tools have real side effects. The model's decisions are not guaranteed.

1.2 Four ways people tried to control it, and why each one leaks

Approach	What it is	Why it is not enough
Prompts	Write the rule in the system prompt.	The model reads it. The model *might* follow it. A prompt is a suggestion, not a wall.
Deny lists	Block dangerous commands by name: rm -rf, DROP TABLE.	The list is never complete. You block rm but miss find -delete. You block DROP but miss TRUNCATE.
IAM / permissions	Decide which user or agent may call which tool.	Controls *who*, not *when*. 'This agent may run bash' does not say 'only after a backup'.
Confirm every call	Ask a human before every tool.	Too noisy. People start clicking yes without reading. Safe tools get the same friction as dangerous ones.

Look at the pattern. Every approach is either a **wish** (prompts) or a **list of names** (deny lists, IAM). None of them has a **memory of what happened already**. The rule 'back up before delete' is about **order**. Order is a kind of state. You cannot enforce an order rule unless something remembers that the backup happened.

1.3 Where should the rule live?



The gate sits between the model and the tool. It is the only place that can say NO and remember what already happened.

Three places a rule can live. PetriFlow puts it in the middle, between the model and the tool.

PetriFlow's answer: put a **gate** between the model and the tool. The gate has two jobs. First, it remembers what has happened so far in this session. Second, it decides allow or block for each tool call using that memory. The model cannot skip the gate, because the gate is not a sentence the model reads. It is code that wraps the tool.

> **WHY THIS DECISION** **Why a gate and not a smarter prompt?**
>
> The author's position (site/src/pages/index.astro, the 'problem' section) is that prompts do not enforce, deny lists are never complete, and IAM controls who but not when. A gate is the only one of the four options that can both remember state and physically refuse.

1.4 The second problem: can you trust the gate?

A gate written as ordinary if-statements has its own risk. Someone adds a code path that forgets to check. A race condition lets two calls through. You test it with ten scenarios and ship, but the eleventh scenario breaks it.

So the author added a second demand: the gate's rules must be **checkable by a machine before the agent starts**. Not 'we tested some cases' but 'we enumerated every possible state and none of them breaks the rule'. That demand is what pushed the design toward Petri nets, which are the subject of the next chapter.

2. Petri nets for a complete beginner

2.1 Cups, marbles, and machines

A Petri net is a board game with three kinds of pieces:

- **Places** are cups. Drawn as circles.
- **Tokens** are marbles that sit in cups. Drawn as black dots. A cup can hold zero, one, or many marbles.
- **Transitions** are machines. Drawn as boxes. Arrows (called **arcs**) connect cups to machines and machines to cups.

There is exactly one rule of play. A machine may **fire** only if every cup with an arrow pointing *into* the machine has at least one marble. When it fires, it takes one marble out of each input cup and drops one marble into each output cup.

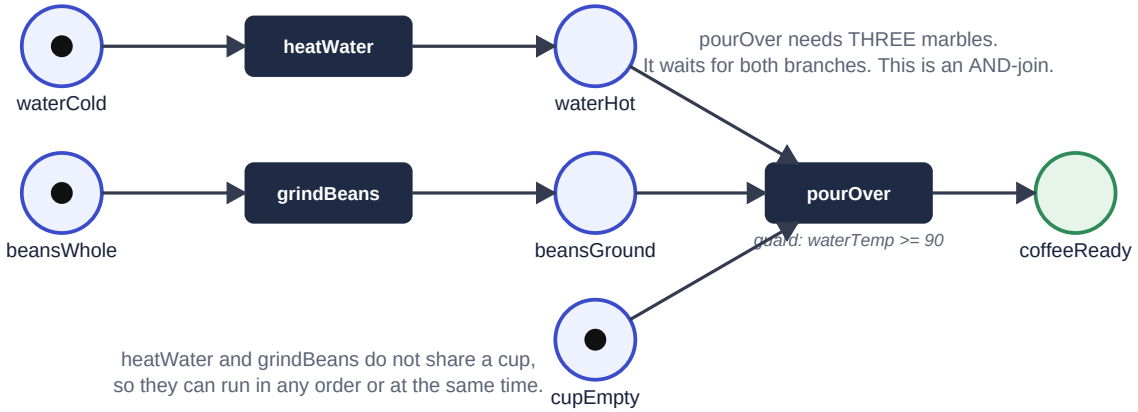


One firing. The marble moves from A to B. Now 'machine' cannot fire again, because A is empty.

The snapshot of how many marbles are in every cup is called the **marking**. In code it is just an object like `{ A: 0, B: 1 }`. The marking is the whole state of the game. Nothing is hidden.

2.2 Doing two things at once: the coffee net

The repo ships a coffee example in workflows/coffee/definition.ts. It is the best first net to look at.

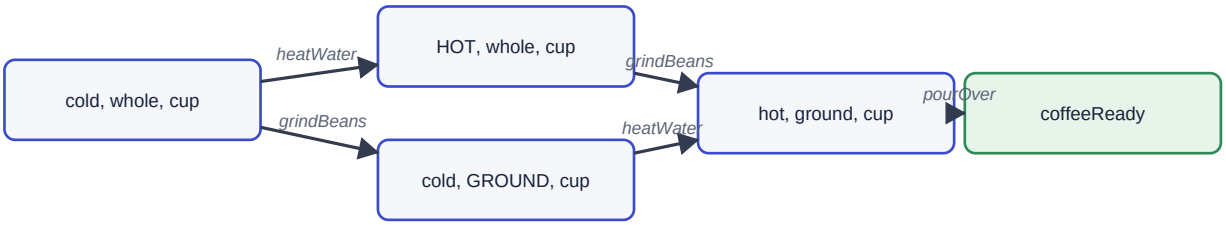


The coffee net. Two independent branches, then a machine that needs all three marbles at once.

Notice two things. **heatWater** and **grindBeans** do not share any cup, so neither blocks the other. They are *concurrent*. **pourOver** has three input arrows, so it waits until water is hot, beans are ground, and the cup is there. That is an *AND-join*. A simple flowchart cannot express 'wait for both' this cleanly. A Petri net does it with one box.

2.3 Every possible future: reachable states

Because the rules are so simple, a computer can play every possible game. Start at the first marking. Try every machine that can fire. Each one gives a new marking. Repeat until nothing new appears. The set you collect is called the **reachable states**.



Every box is one 'marking' (a snapshot of all cups). The analyser walks every arrow from the start and collects every box it can reach. Here: 5 reachable states, 1 terminal state, and it is the good one.

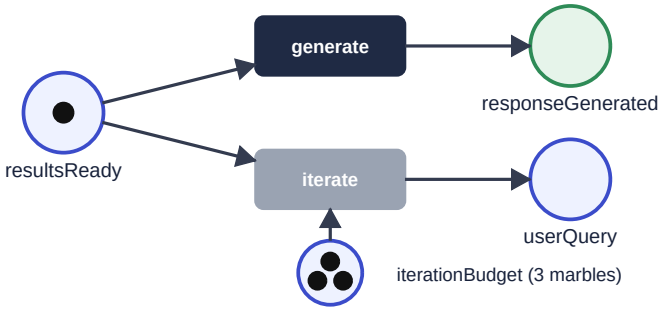
The reachability graph of the coffee net. Five snapshots, one finish.

From this graph you can answer hard questions with certainty:

- **Can it get stuck?** A state where no machine can fire is a **terminal state**. If a terminal state is not one you wanted, that is a **deadlock**.
- **Can something bad ever happen?** Check every state in the list. If none has the bad pattern, it is impossible, not just unlikely.
- **Is something conserved?** If 'inventory + reserved + shipped' is 3 in every state, you can never oversell. This is called an **invariant**.

> **IN SIMPLE WORDS Why this is stronger than testing**
>
> A test runs one path and checks the end. Reachability analysis runs **every** path. For small nets this takes milliseconds. The agent benchmark net in this repo has 16 cups and 17 machines, and only 196 reachable states out of 65,536 possible combinations. The shape of the net rules out 99.7 percent of states before anything runs.

2.4 Choice and budget



CHOICE: two machines want the same marble. Only one wins.

BUDGET: 'iterate' also eats one budget marble. After 3 loops the budget cup is empty, so only 'generate' can fire. Loop is bounded.

Two more patterns from workflows/simple-agent: a choice point and a budget cup that bounds a loop.

A **choice** is two machines that want the same marble. Only one can take it. Who decides? In the workflow engine, a *decision provider* (often an LLM) picks. In the gate, the agent's tool call picks. A **budget** is a cup pre-filled with N marbles that a looping machine must also eat from. When the cup is empty, the loop machine can no longer fire. The loop is bounded by shape, not by a counter variable someone might forget to check.

2.5 Why not a state machine or a DAG?

Model	How many 'current positions'?	Can it count?	Can it loop?	Good for
Finite state machine	Exactly one	No	Yes	One thing at a time
DAG (n8n, Airflow style)	Many, but only forward	No	No	Pipelines with no cycles
Petri net	Many marbles, anywhere	Yes (marble counts)	Yes, and bounded by budgets	Concurrency, resources, ordering rules

The CLAUDE.md file in the repo says it directly: PetriFlow 'uses Petri nets (not DAGs) as the core primitive, enabling formal verification of concurrent constraints, resource contention, and multi-step safety policies before an agent runs'.

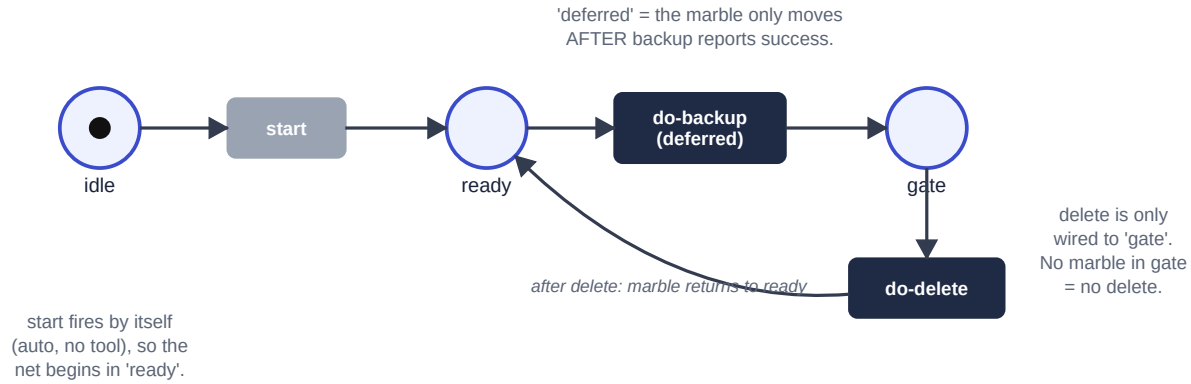
3. The big trick: a rule becomes a shape

Here is the central idea of the whole repository. Instead of writing a rule as an if-statement, write it as a tiny Petri net. Each tool the agent might call is attached to one machine in the net. The agent may call the tool only if that machine can fire right now. The shape of the net *is* the rule. Since the net is small, every state can be enumerated, so the rule is verified before the agent starts.

The rules package (packages/rules/src/compiler.ts) knows five sentence patterns. Each one compiles to one fixed net template. Let us look at every template.

3.1 require A before B

```
require backup before delete
```



compileSequence(): three cups, three machines. Reachable states: 3.

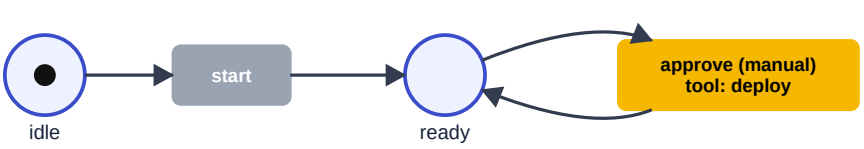
Walk it through. The net starts with a marble in *idle*. The *start* machine is **auto** and has no tool attached, so the gate fires it immediately (this is called *autoAdvance*, see chapter 6). Now the marble is in *ready*. The only machine that can fire is *do-backup*. The *do-delete* machine is wired to *gate*, which is empty. So delete is blocked.

When the agent calls backup, the gate allows it, but because the machine is **deferred**, the marble does not move yet. Only when the backup tool reports success does the marble move to *gate*. Now delete can fire. After delete, the marble goes back to *ready*. A second delete needs a second backup.

> **WHY THIS DECISION** **Why does the marble go back to 'ready' after delete?**
>
> Because the rule is meant to protect *every* delete, not just the first. If the marble stayed in 'gate', one backup would unlock unlimited deletes. The loop arrow from do-delete to ready is what makes the rule say 'before each delete', which is the safer reading.

3.2 require human-approval before B

```
require human-approval before deploy
```



manual = ask a human every time. Marble returns, so it can be asked again.

compileApproval(): a manual machine on a self-loop. Reachable states: 2.

A **manual** machine means: before firing, call `ctx.confirm()` and ask a human. If the human says no, block. If the environment has no user interface at all (`hasUI: false`), the gate blocks too. This is a 'fail closed' choice: when nobody can approve, the answer is no.

3.3 block A

```
block rm
```



No arrow ever puts a marble into 'locked'. The 'rm' machine is wired only to that empty cup. The analyser can prove it: in all 2 reachable states, 'locked' is 0, so 'do-rm' is a dead transition.

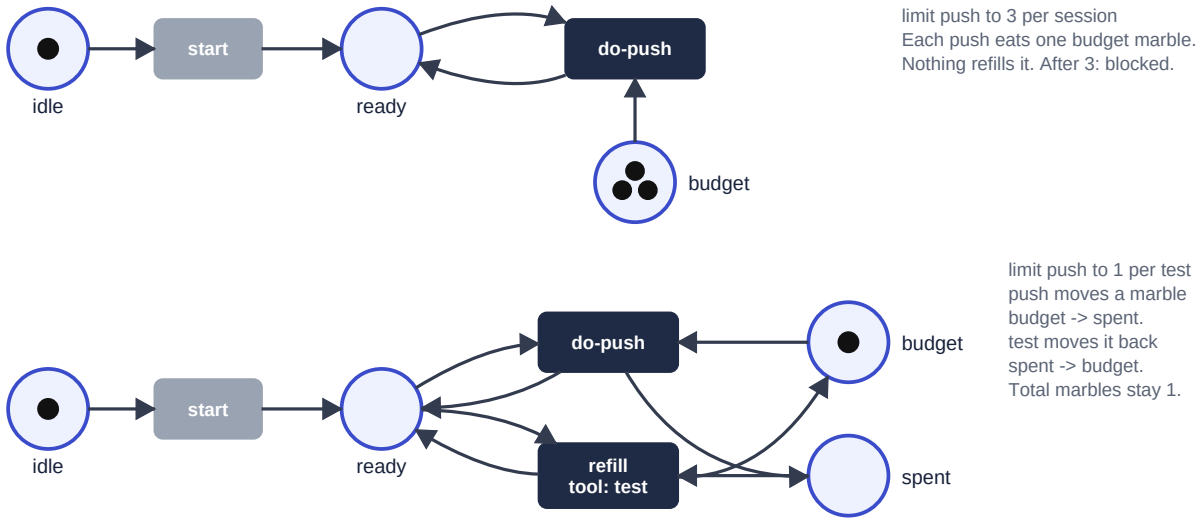
compileBlock(): the tool's machine is wired to a cup that can never receive a marble.

This is the most elegant template. There is no if-statement that says 'deny rm'. The *do-rm* machine simply needs a marble from *locked*, and no arrow in the whole net ever puts a marble there. The analyser calls this a **dead transition**. It is provably unreachable.

> **WHY THIS DECISION** **Why not just a list of forbidden names?**
>
> A forbidden-name list and a dead transition behave the same at runtime. The difference is that the dead transition is *one more net* in the same system, so it composes with every other rule, shows up in the same status output, and is proved by the same analyser. One mechanism instead of two.

3.4 limit A to N per session / limit A to N per action

```
limit push to 3 per session
limit push to 1 per test
```



compileLimit(): a budget cup. The 'per action' form adds a 'spent' cup and a refill machine.

The session form starts with N marbles in *budget*. Each use eats one. When the cup is empty, the machine cannot fire. The per-action form keeps the total number of marbles constant: a use moves one marble from *budget* to *spent*, and the refilling tool moves it back. Token conservation gives a free invariant: budget + spent = N, always.

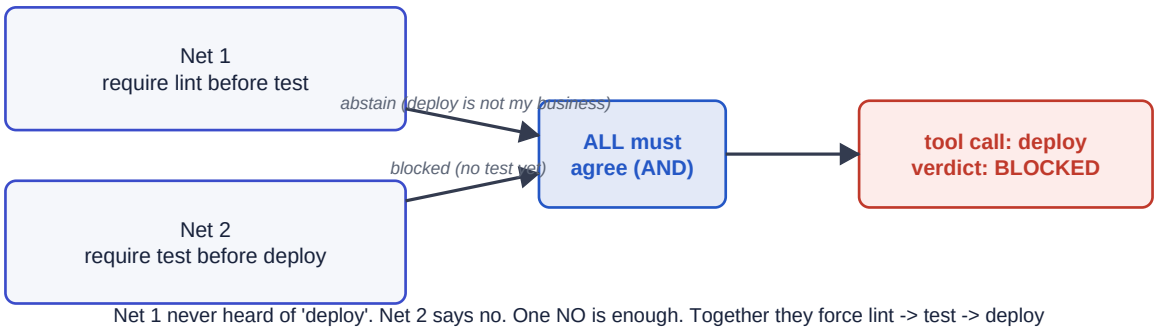
3.5 map tool.field pattern as name

```
map bash.command rm as delete
map bash.command cp as backup
require backup before delete
```

Many real agents have one big tool called *bash*. The rules above cannot see inside it. A **map** line teaches the compiler: when the tool is bash and its *command* field matches the word *rm*, pretend the tool was called *delete*. Bare words get automatic word boundaries, so *rm* matches *rm -rf build/* but not *format*. A pattern wrapped in slashes is a raw regular expression.

There is also **dot notation**. Writing *discord.sendMessage* in any rule means: tool name is *discord* and the input has `action: "sendMessage"`. The compiler generates the mapping function automatically. Actions not named in any rule pass freely.

3.6 Rules compose by intersection



Net 1 never heard of 'deploy'. Net 2 says no. One NO is enough. Together they force lint -> test -> deploy without either net knowing about the other. Safety rules compose by intersection.

Two independent nets. Neither knows the other exists. Together they enforce a three-step order.

This is the second big design decision. Every rule is its own net, and the manager asks **all** of them. One net saying 'blocked' is enough to block. This is exactly how safety properties behave in mathematics: if property A holds and property B holds, then A-and-B holds. There is no need to build one giant net that knows every rule, and no risk of the state space exploding when you add a rule. Each net stays at 2 to 5 states, and verification stays instant.

> **WHY THIS DECISION** **Why many tiny nets instead of one big one?**
>
> The rules README explains it: 'each net is small enough to verify exhaustively (a few reachable states), but their combined enforcement covers complex multi-step policies. You get compositional guarantees without a combinatorial explosion.' The cost is that no single analysis looks at the combination. Chapter 10 returns to what that costs.

3.7 What 'verified at compile time' means for these rules

When you call `compile()`, the function `verifyNets()` runs the petri-ts analyser on each net and records the number of reachable states. That is the whole verification step in the rules package. It proves that the net is finite and well-formed. It does **not** check a named property such as 'delete never fires before backup'. That property is true because the template's shape makes it true, and because the templates were designed by hand. Keep this in mind for chapter 10.

4. The map of the repository

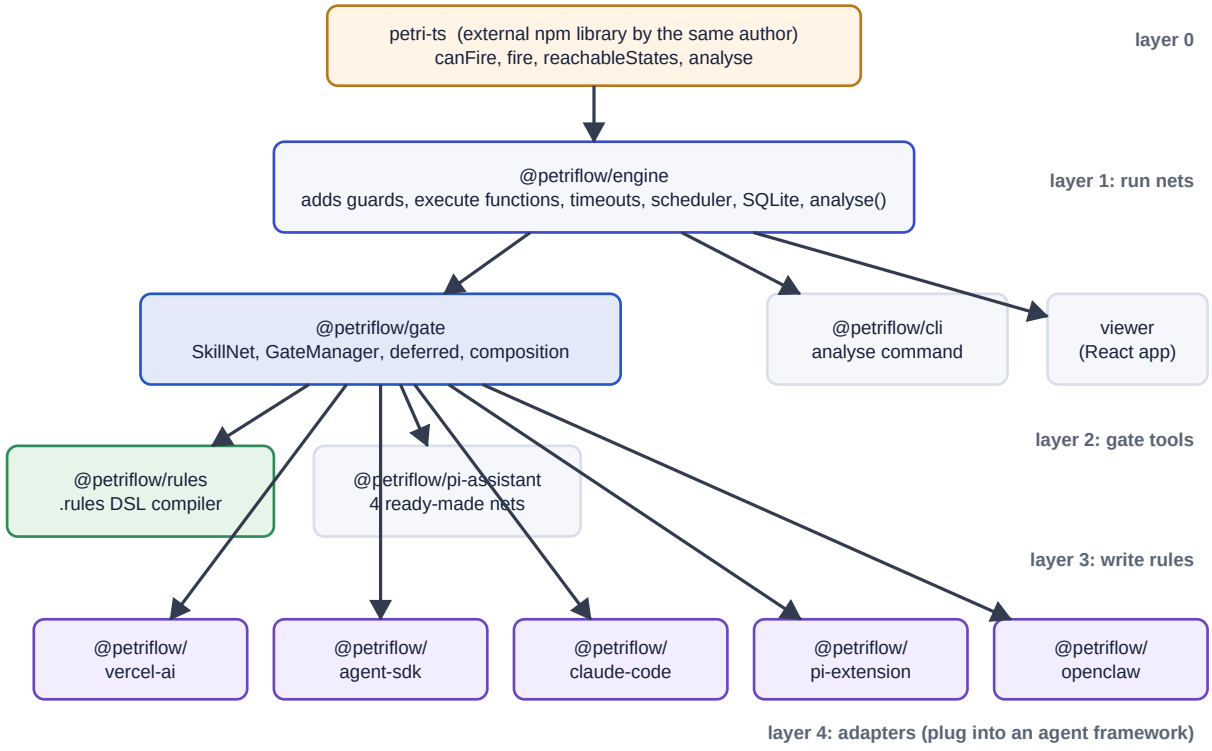
4.1 Two products in one repo

The README is honest about this: the repository contains **two related but separate things**.

- **The safety layer** (the main product): gate, rules, and the adapters that plug into agent frameworks. This is what chapters 1 to 3 described.
- **The workflow runtime** (the older, more general thing): an engine that runs any Petri net workflow with side effects, timeouts, a scheduler, and a database. Plus a CLI that analyses workflows and a React viewer that lets you click on transitions.

The safety layer is built on top of the workflow engine's types, but it uses only a small part of them (canFire, fire, and the Marking type).

4.2 The layer diagram



Package dependencies. Arrows point from a package to the packages that depend on it. Source: CLAUDE.md and each package.json.

4.3 Folder by folder

Folder	What is inside, in plain words
packages/engine	Core. Types for places and transitions. Guards (small expressions), execute functions, timeouts. A tick-based Scheduler. A SQLite adapter. analyse().
packages/gate	The safety core. SkillNet type, the 4-phase tool call handler, deferred transitions, autoAdvance, GateManager for many nets, replay.
packages/rules	The .rules text language. A parser, five net templates, automatic tool mapping, state counting.

Folder	What is inside, in plain words
packages/vercel-ai	Adapter for the Vercel AI SDK. Wraps each tool's execute() with the gate. Rebuilds state from message history.
packages/agent-sdk	Adapter for the Claude Agent SDK. Returns hook callbacks that run in the same process.
packages/claude-code	Adapter for the Claude Code CLI. A script that runs once per hook event and keeps state in a JSON file.
packages/pi-extension	Adapter for pi-coding-agent. Listens to tool_call and tool_result events, adds /net-status commands.
packages/openclaw	Adapter for OpenClaw. Same idea, but must invent tool call ids because OpenClaw does not provide them.
packages/pi-assistant	Four hand-written nets: cleanup (backup before destroy with path matching), communicate, deploy, research.
packages/cli	petriflow analyse workflow.ts. Prints reachable states, deadlocks, invariants. --strict exits 1 for CI.
apps/viewer	React + xyflow app. Click transitions, watch tokens move, see analysis, edit nets. Expects an API on port 3000 for saving.
workflows/	Six example workflow nets with tests: coffee, github-lookup, contract-approval, order-checkout, simple-agent, agent-benchmark.
examples/	Five folders of runnable Vercel AI agents with .rules files (file management, deployment, Discord bot, DevOps assistant, openclaw-ops).
comparisons/	The same scenarios built in n8n and against OpenClaw's policy model, for the benchmark.
site/	The documentation website (Astro). Deployed to GitHub Pages by .github/workflows/deploy-site.yml.
BENCHMARK.md	PetriFlow vs n8n vs ReAct on one agent. Same LLM call count, but only PetriFlow proves properties.

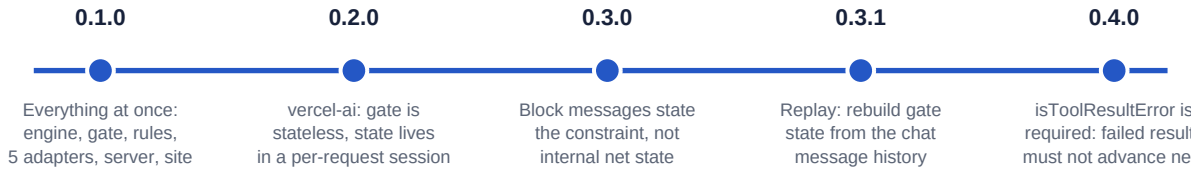
4.4 Tooling choices

- **Bun everywhere.** Package manager, test runner, runtime, and bun:sqlite for the database. CLAUDE.md notes that better-sqlite3 had native binding problems under Bun, so the built-in one was chosen.
- **Turborepo** orders builds. turbo.json says test and check depend on building dependencies first, so you must build before testing.
- **TypeScript strict** with noUncheckedIndexedAccess. That is why you see `matching[0]!` with an exclamation mark in the code.
- **tsup** bundles the published packages (engine, gate, rules, vercel-ai) to dist/ with both ESM and CommonJS outputs.
- **CI** (.github/workflows/ci.yml) runs bun install, bun run build, bun test on every push and pull request to main.

5. How it was made: the story behind the decisions

The folder you have has no git history, so the timeline below comes from CHANGELOG.md, the package versions, and the design notes scattered in README files. Each step is told as problem, then decision.

Each step was a fix for a problem found while using the gate with a real agent framework.



Source: CHANGELOG.md. Versions are for @petriflow/vercel-ai and @petriflow/gate.

Release timeline reconstructed from CHANGELOG.md.

5.1 Before the repo: petri-ts

The author first wrote a small standalone library called **petri-ts**. It knows only the pure game: canFire, fire, enabledTransitions, reachableStates, terminalStates, and analyse. It has no idea about tools, LLMs, or side effects. PetriFlow imports it and never re-implements it. You can see this in packages/engine/src/engine.ts, which is only 80 lines because the hard part is imported.

> **WHY THIS DECISION** **Why keep the pure library separate?**
>
> The analyser must be trusted. Keeping it in a dependency with no side effects means the thing that produces proofs never touches the thing that runs tools. It also lets toNet() strip away guards and execute functions and hand a plain net to the analyser, which is how 'analysis of structure only' is guaranteed.

5.2 0.1.0: the workflow engine, then the gate

Problem: a Petri net that only moves marbles is not useful. Real steps call HTTP, wait for timers, and need a human to click approve. **Decision:** extend each transition with optional *guard* (a boolean expression on context), *execute* (an async function returning a context patch), *timeout* (after N ms, drop a token somewhere else), and *config*. Crucially the extension is a TypeScript *intersection type*: a WorkflowTransition **is** a petri-ts Transition, so every analysis function accepts it unchanged.

Problem: the benchmark showed that modelling an agent as a workflow proves termination and human gates, but most people do not want to rewrite their agent as a net. **Decision:** flip the relationship. Let the agent framework stay in charge of the loop, and put a net *next to* it that only answers 'may this tool be called now'. That is the gate package. A SkillNet is a Petri net whose transitions carry a `tools[]` list.

Problem: a tool call is allowed, the net advances, and then the tool fails. Now the net thinks a backup exists when it does not. **Decision: deferred transitions**. Allow the call, remember it as pending, and only fire when the result comes back with no error.

Problem: writing nets by hand is error-prone and most safety rules are the same five sentences. **Decision:** the rules DSL. Five templates, each a verified net, composed by AND.

5.3 0.2.0: the gate must be stateless

Problem: in a web server, one gate object was shared across many users' requests. State leaked between sessions. **Decision:** `createPetriflowGate()` holds no state. Each `wrapTools()` call creates a fresh GateManager and returns a *GateSession*. This is a breaking change and the changelog says so.

5.4 0.3.0: block messages the model can act on

Problem: the block message was 'Tool deploy not available in current state. Marking: ready:1'. The LLM read that and gave up. **Decision:** every compiled net carries *ruleMetadata*, and `formatBlockReason()` turns it into a plain sentence: 'deploy requires a successful call to test first.' The model now knows what to do next. This is a nice example of designing for a non-human reader.

5.5 0.3.1: replay from history

Problem: the Vercel AI SDK is stateless. Each request arrives with the whole chat history and nothing else. Where does the gate state live between requests? **Decision:** derive it. `extractReplayEntries()` walks the messages, finds every tool result, and `manager.replay()` re-fires the matching transitions. The chat log becomes the source of truth. The code comment warns: use server-stored messages, never client-supplied ones.

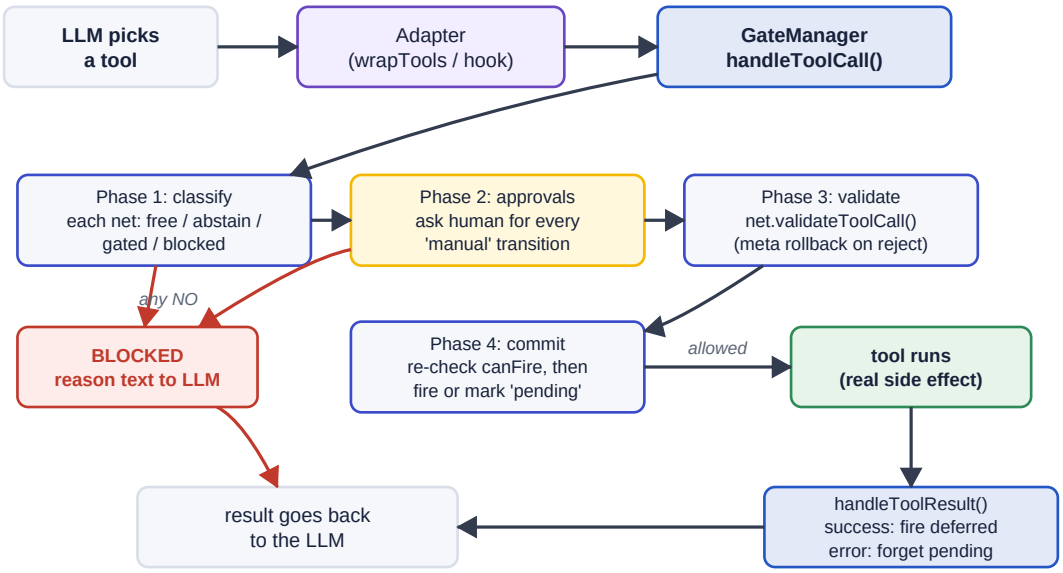
5.6 0.4.0: 'failed' is not always an exception

Problem: the AI SDK converts thrown errors into ordinary return values. A backup that returned `{success: false}` looked like success, and deferred transitions fired anyway. **Decision:** make `isToolResultError` a required option. Users must say what failure looks like for their tools. If the callback itself throws, the result is treated as an error. Fail closed again.

> **NOTE A pattern in all of these**
>
> Every version after the first was a reaction to running the gate inside a real framework and discovering an assumption that did not hold: shared state, unhelpful messages, no state between requests, errors that look like results. The core net logic barely changed. The edges did.

6. Inside the gate: one tool call's journey

This chapter follows a single tool call through packages/gate. The main file is src/compose.ts (composedToolCall) with helpers in src/gate.ts, src/advance.ts and src/manager.ts.



The life of one tool call. Four phases, two exits.

6.1 Phase 1: every net gives a verdict

`classifyNets()` asks each active net one question and gets one of four answers. Think of a panel of judges.

Verdict	The judge says	How the code decides
free	'That tool is always fine with me.'	The resolved tool name is in net.freeTools.
abstain	'Not my business.'	No transition in this net lists the tool at all (no jurisdiction).
gated	'Yes, and I will record it.'	At least one enabled transition lists the tool. The first match is kept.
blocked	'No.'	Some transition lists the tool, but none of them can fire right now.

Decision rule: one **blocked** wins immediately. If nobody blocks and nobody is gated, the call passes without touching any net. Otherwise the gated nets continue to phase 2.

> **IN SIMPLE WORDS Abstain is what makes composition work**
>
> If a net blocked every tool it had not heard of, you could never combine two nets. 'Not my business' lets ten small nets coexist, each watching only its own tools.

6.2 Phase 2: human approvals

For every gated net whose transition is **manual**, the gate calls `ctx.confirm(title, message)` and waits. No UI means an automatic no. A human no means block with the reason 'rejected by human review'.

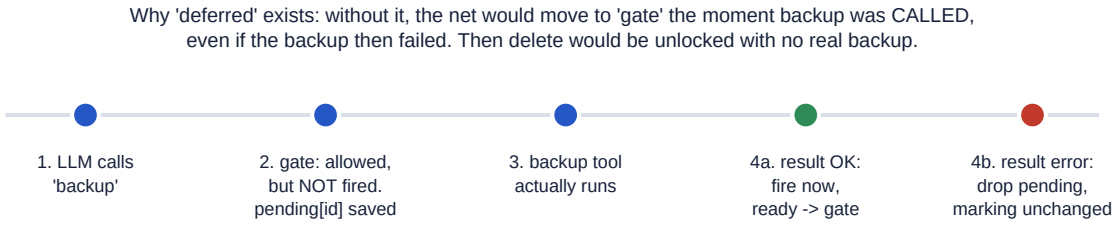
6.3 Phase 3: semantic validation with rollback

A net may define `validateToolCall()` for checks the shape alone cannot express, such as 'the backup must cover the same path as the delete'. Validators may write into `state.meta`. If a later validator rejects, earlier writes would be half-applied, so the code snapshots every net's meta with structuredClone before the loop and restores all of them on rejection.

6.4 Phase 4: commit, with a second look

Phases 2 and 3 contained *await*. While the gate waited for a human, another tool call might have changed the marking. So before firing, every non-deferred transition is re-checked with canFire. The tests call this the TOCTOU guard (time of check to time of use). Then: deferred transitions are stored in `state.pending` under the toolCallId, and all others fire immediately, followed by autoAdvance.

6.5 Deferred transitions in detail



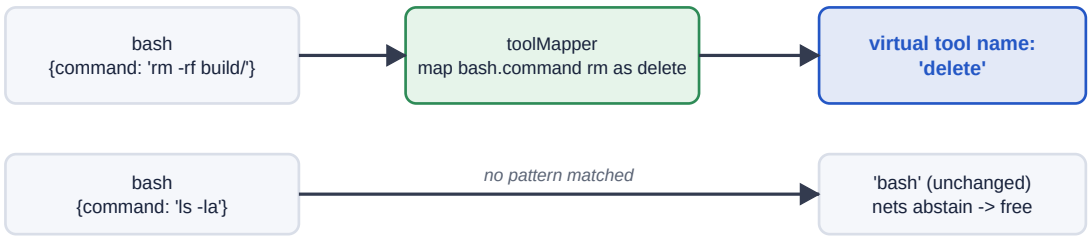
The deferred timeline. The net only believes success it has seen.

`handleToolResult()` looks up the pending entry by toolCallId. On error it deletes the entry and does nothing else. On success it fires, calls `onDeferredResult()` so the net can record metadata (the cleanup net stores backed-up paths here), then autoAdvances.

6.6 autoAdvance: the net moves itself when it can

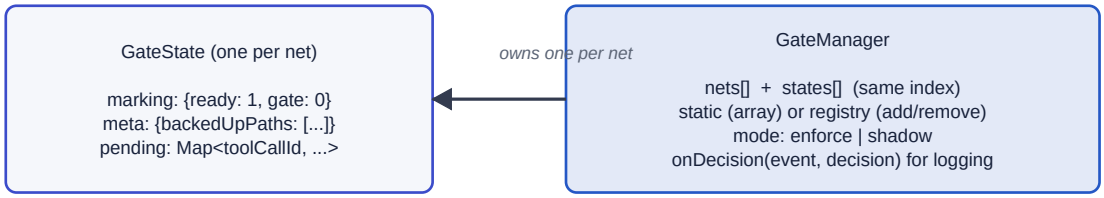
A transition that is **auto** and has no tools list is called *structural*. It represents the net's own bookkeeping, like the *start* machine in every template. `autoAdvance()` (src/advance.ts) fires all enabled structural transitions in a loop until nothing changes. Two safety rules: if two structural transitions compete for the same marble, neither fires (no silent arbitrary choice), and a seen-markings set stops infinite cycles.

6.7 Tool mapping



resolveTool(): the physical tool name becomes a virtual name before any net looks at it.

6.8 The manager and its state



GateManager holds one GateState per net. Everything the gate knows is in these three fields.

- **Static mode**: pass an array of nets. All are always active.
- **Registry mode**: pass `{registry, active}`. Nets can be switched on and off at runtime with addNet / removeNet. A removed net keeps its marking, and tool results still reach it so pending deferreds can resolve.
- **Shadow mode**: the manager computes the decision and reports it through onDecision, but never blocks. This is how you trial rules on live traffic safely.
- **replay(entries)**: for each successful past tool call, find the matching enabled transition and fire it. Failed entries are skipped. Already-advanced nets skip silently.
- **formatSystemPrompt()**: a Markdown block per net listing available gated tools, free tools, and the current marking. Adapters inject it into the LLM's system prompt so the model knows the rules before it acts.

7. Inside the rules compiler



packages/rules/src/compiler.ts, function compile(). Map lines are collected first, then applied to every rule net.

From text to verified nets in five steps. All in one file, about 480 lines.

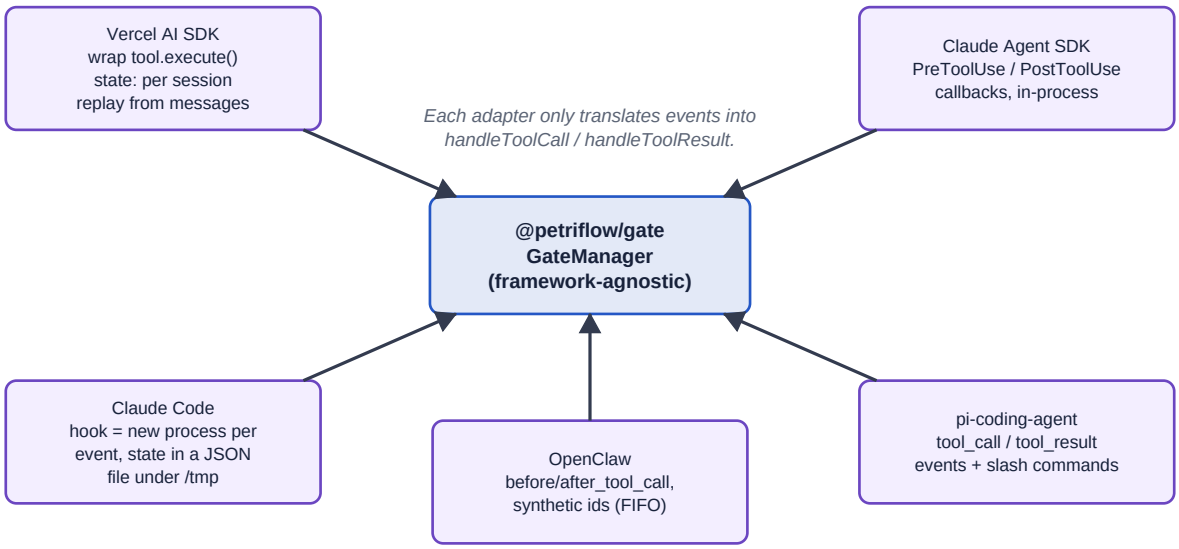
The parser is deliberately dumb. It splits each line on whitespace and counts tokens. 'require A before B' must be exactly four tokens. Any deviation throws an error with the line number. There is no grammar library, no AST, no precedence. This keeps the language tiny enough that a reader can hold all of it in their head.

Order of operations inside `compile()`: first every line is parsed, and *map* lines are set aside. Then each rule becomes a net from its template. Then `buildToolMapper()` attaches a mapper to each net if the net uses dotted names or if any map lines exist. Map patterns are checked first because they are more specific. Finally `verifyNets()` counts states.

Each compiled net also carries `ruleMetadata` so the gate can say 'push has reached its limit of 3 calls per session' instead of printing a marking.

> **WHY THIS DECISION** **Why a text DSL and not just TypeScript objects?**
>
> Both exist. defineSkillNet() lets you write any net in TypeScript. The DSL covers the common cases in one line each, is readable by people who are not programmers, and can be loaded from a .rules file at startup. The README of @petriflow/rules shows both composed together in one manager.

8. Adapters: plugging into real agent frameworks



One core, five adapters. The core never imports a framework.

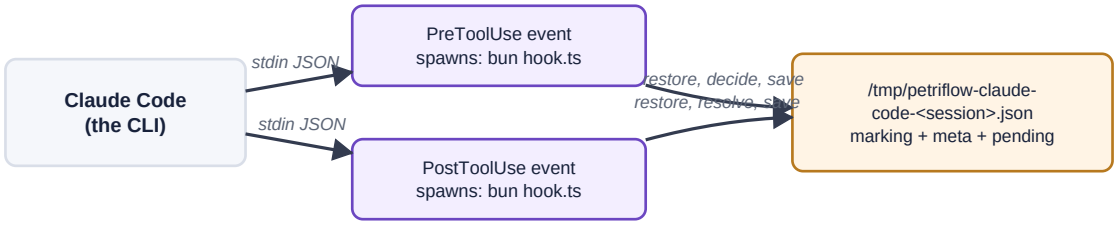
Adapter	Where it hooks in	Where state lives	Human approval?	How failure is detected
vercel-ai	Wraps every tool's execute()	Per wrapTools() session, or rebuilt from messages	Yes, if a confirm callback is passed	Thrown error, or the required isToolResultError callback
agent-sdk	PreToolUse / PostToolUse / PostToolUseFailure callbacks	In the process, one manager per gate	Yes, if confirm is passed	PostToolUseFailure event
claude-code	Same three hook events, but as a shell command	JSON file under /tmp, keyed by session id	No (hasUI is false)	PostToolUseFailure event
pi-extension	tool_call and tool_result events	In the process	Yes, via ctx.ui.confirm	event.isError
openclaw	before_tool_call / after_tool_call	In the process	No (hasUI is false)	event.error present

8.1 Vercel AI SDK: the reference adapter

`wrapTools()` (src/wrap-tools.ts) replaces each tool's execute with a function that first asks the manager, throws `ToolCallBlockedError` if blocked, otherwise runs the real tool and then reports the result. Tools without execute pass through untouched. If you pass `{ messages }`, the session first replays history so a stateless server request starts with the right marking.

```
const gate = createPetriflowGate(nets, {
  isToolResultError: (name, result) => (result as any)?.success === false,
});
const session = gate.wrapTools(myTools, { messages });
await generateText({ tools: session.tools, system: session.systemPrompt(), messages });
```

8.2 Claude Code: a process that forgets everything



The hook process has no memory between events, so ALL state is written to disk and read back every time.
Output JSON with permissionDecision: 'deny' tells Claude Code to refuse the tool.

The claude-code adapter. Every event is a fresh process, so state must round-trip through a file.

src/hook.ts reads one JSON event from stdin. On SessionStart it deletes the state file. On PreToolUse it rebuilds a manager, restores state from the file, decides, saves, and prints a deny decision if needed. On PostToolUse it restores, resolves pending deferreds, saves. Without a config file, it uses the bundled *safe-coding* net: Read, Glob, Grep and WebSearch are free, Write, Edit, WebFetch and Task are gated, Bash is dead.

8.3 OpenClaw: inventing ids

OpenClaw's hooks do not pass a tool call id, but deferred transitions need one to match a result to a call. The adapter generates 'gate-1', 'gate-2' and keeps a first-in-first-out queue per tool name. When a result arrives for tool X, it is matched to the oldest unanswered call of X. This works when calls of the same tool finish in order.

8.4 pi-assistant: the smart hand-written nets

The cleanup net (packages/pi-assistant/src/nets/cleanup.ts) shows what the DSL cannot do. Its mapper recognises many backup commands (cp -r, tar, git stash, pg_dump, rsync) and many destructive ones (rm, git reset --hard, DROP TABLE, TRUNCATE). It extracts the *path* each one touches. On a successful backup it records the path. On a destructive call its validateToolCall checks that some recorded backup *covers* the target, and then consumes that backup entry. This is the only place in the repo where 'backup before delete' means 'backup of the same thing'.

9. The other half: the workflow engine

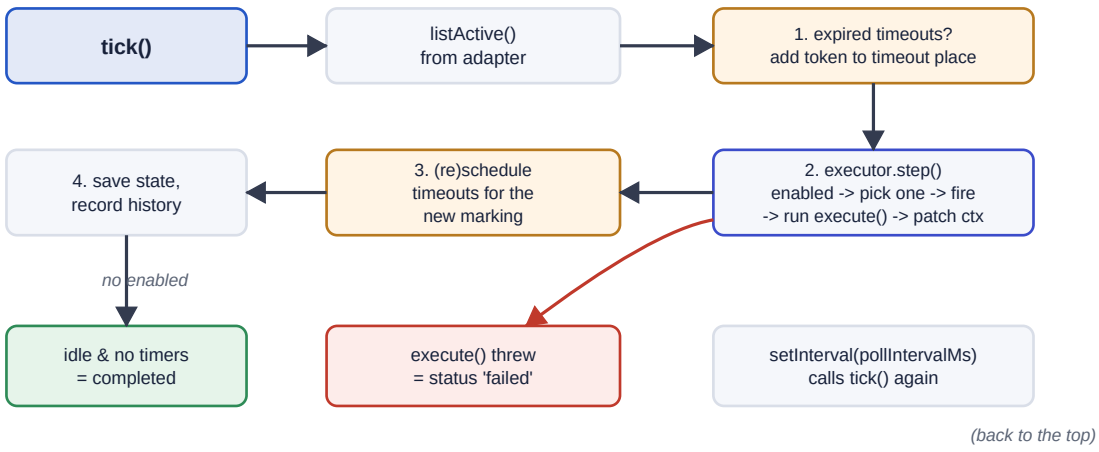
The engine runs whole workflows as nets, with real side effects. It is the older part of the project and the base the gate is built on. Main files: packages/engine/src/workflow.ts, executor.ts, scheduler.ts, persistence/sqlite-adapter.ts, analyse.ts.

9.1 Defining a workflow

`defineWorkflow()` takes places, transitions, an initial marking, an initial context object, terminal places, and optional invariants. It validates that every arrow points at a known place, compiles each guard string with the **filtrex** expression library (so guards are data, not code, and can be stored), and pulls each execute function into a separate map. Transitions with a *type* of http or timer and a *config* get an executor from a small node registry. The http node refuses private and loopback addresses, which is a server-side request forgery guard.

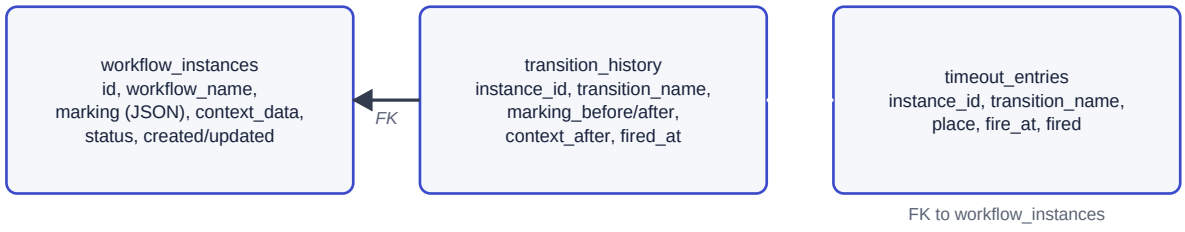
9.2 Running: executor and scheduler

`createExecutor().step()` lists transitions that are both structurally enabled and pass their guard. If more than one is enabled and a *decision provider* is configured, the provider (often an LLM) chooses and explains why. Then the transition fires, its execute runs, and the returned patch is merged into context. `stepBatch()` fires several non-conflicting transitions at once with Promise.all and merges patches in alphabetical name order.



One Scheduler.tick(). Everything goes through the persistence adapter, so a crash loses at most one tick.

9.3 Persistence

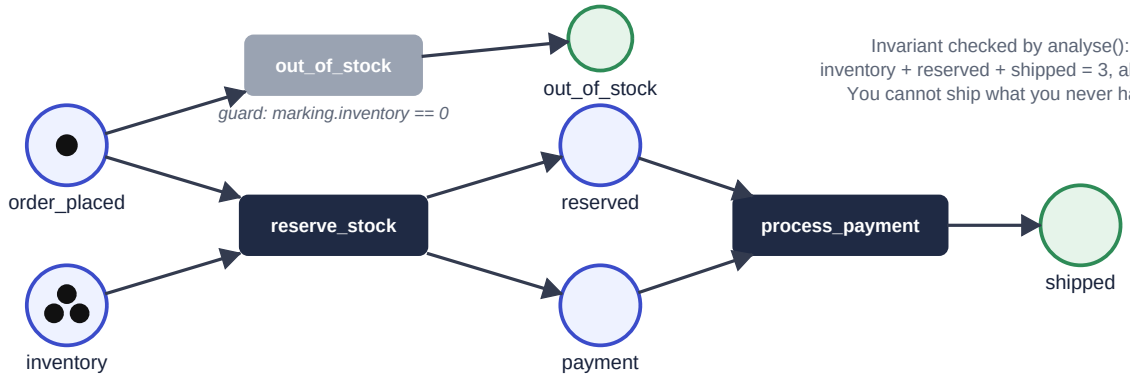


Three SQLite tables. The adapter interface is small enough to re-implement for Postgres or Redis.

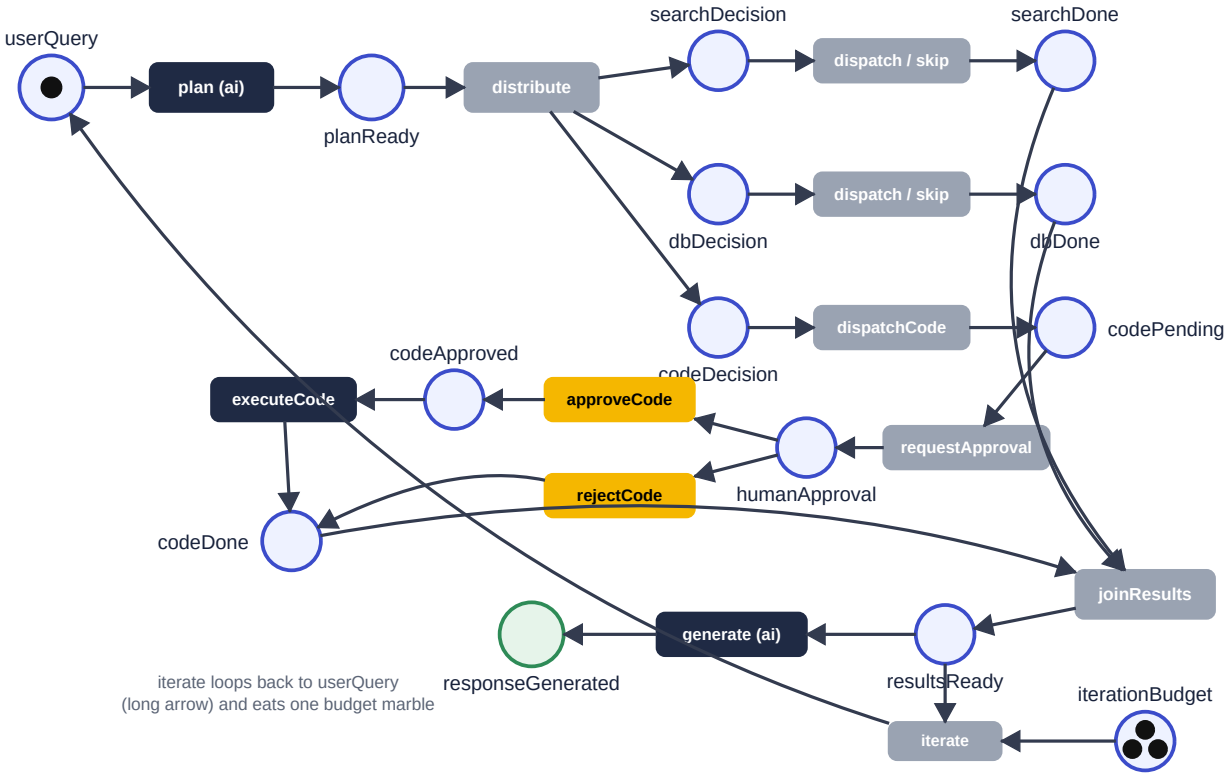
9.4 Analysis and the CLI

`analyse(definition)` calls `toNet()` to strip guards and executors, runs the petri-ts analyser, then splits terminal states into *valid* (a terminal place has a token) and *unexpected* (stuck somewhere else). The CLI prints this and, with --strict, exits 1 on unexpected terminals or broken invariants. That makes 'the workflow cannot deadlock' a CI check.

9.5 Two example nets worth studying



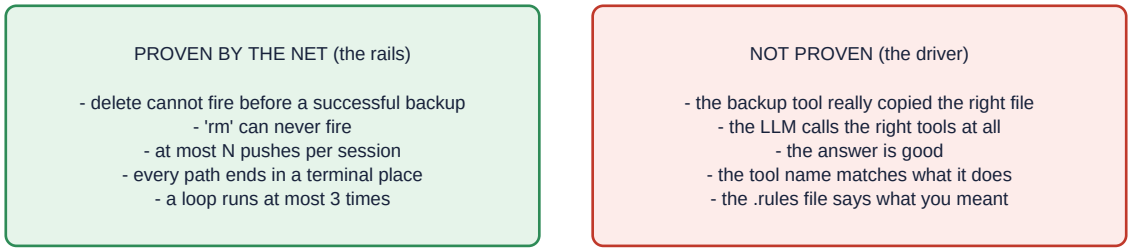
workflows/order-checkout. A resource pool and an invariant that proves you cannot oversell.



workflows/agent-benchmark, simplified. Fan-out to three tools, a human gate on code, an AND-join, and a bounded loop.

The benchmark test file (agent-benchmark.test.ts) proves four properties over all 196 reachable states: every terminal state is responseGenerated, no transition goes from codePending to codeDone without passing humanApproval, joinResults needs all three done tokens, and iterationBudget stays between 0 and 3. BENCHMARK.md compares this with n8n (20 nodes, nothing proven) and a plain ReAct loop (11 LLM calls instead of 2).

10. What 'proven' really means here



The net is the specification. If the shape is wrong, the proof is still correct but useless.

The rails are proven. The driver is not. BENCHMARK.md says this itself.

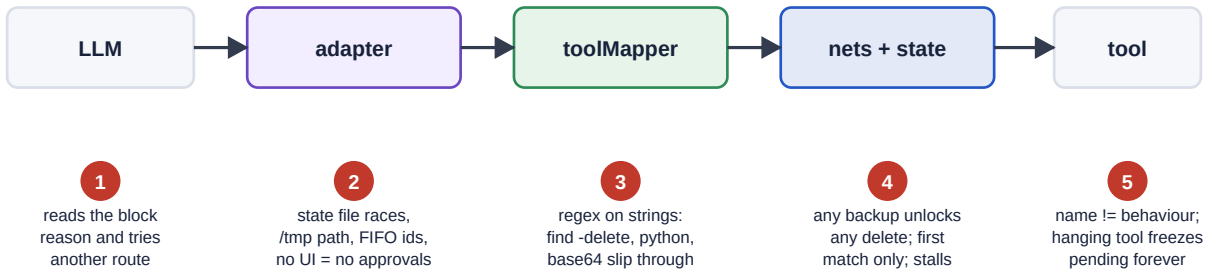
Three facts about the proofs that the README's headline ('rules your agent cannot break') does not spell out:

- **Analysis looks at structure only.** toNet() removes guards before analysing. A guard that is always false would make a transition dead at runtime, but the analyser still counts states as if it could fire. So the analysis *over-approximates* reachable states. That is safe for 'bad thing never happens' claims (if it cannot happen even in the bigger set, it cannot happen in the smaller one), but *unsafe* for 'it always finishes' claims: a guard can create a deadlock the analyser cannot see.
- **The rules package only counts.** compile() returns reachable state numbers. It never asserts a property. The template shapes are what carry the guarantee, and they were designed by hand.
- **The composition is never analysed as a whole.** Each tiny net is enumerated alone. Two nets that are each deadlock-free can still jointly block everything (for example 'require A before B' plus 'require B before A'). Nothing in the repo detects that; the agent would simply find every tool blocked.

> **IN SIMPLE WORDS** **The right mental model**
>
> PetriFlow proves statements about **which tool names may be called in which order**. It proves nothing about what the tools do, what the model intends, or whether your rule file expresses the policy you had in mind.

11. What it is not telling you: loopholes

Everything below was found by reading the code, not by running attacks. Each item names the file so you can judge it yourself. Severity is a judgement: **high** means a rule can be bypassed, **medium** means a rule can silently stop working, **low** means rough edges.



Numbers refer to the list in chapter 11.

Where the holes sit on the tool call journey.

Group 1: the gate sees names, not reality

- **[high] A tool's name is trusted as its meaning.** 'require backup before delete' is satisfied by any tool called backup that returns without error. If the backup tool is buggy, backs up the wrong thing, or is a stub, the net still unlocks delete. The gate cannot know. (compiler.ts compileSequence; gate.ts handleToolResult)
- **[high] Any backup unlocks any delete in the DSL.** The sequence template has no idea of *what* was backed up. Backing up readme.txt unlocks deleting the database. Only the hand-written cleanup net in pi-assistant checks path coverage. The DSL has no syntax to ask for it.
- **[high] Tool mapping is a regular expression on a string.** 'map bash.command rm as delete' catches rm but not find -delete, shred, unlink, python -c 'os.remove', a shell variable like $CMD, or a base64 decode piped to sh. The site's own problem statement ('deny-lists are never complete') comes back inside the map layer. (compiler.ts parsePattern)
- **[medium] 'block rm' only blocks a tool literally named rm.** Example 01 works because rm is a separate tool. With a real bash tool the block needs a map line, and then item 3 applies.

Group 2: state can be wrong or lost

- **[high] Replay trusts the message history.** extractReplayEntries() treats every tool-result part as true. If a client can send its own message history, it can include a fake 'backup succeeded' result and arrive with delete already unlocked. The code comment warns about this, but nothing enforces it. (vercel-ai/src/replay.ts)
- **[high] Examples set isToolResultError to always false.** examples/01-file-management/agent.ts passes `() => false`. A tool that returns `{ok: false}` instead of throwing would advance a deferred transition. The 0.4.0 fix only helps if users write the callback correctly.
- **[medium] A hanging tool freezes a deferred transition forever.** wrap-tools.ts says so in a comment. There is no timeout and no cleanup of stale pending entries.
- **[medium] Claude Code state file races.** hook.ts does read, decide, write on every event. Claude Code can run several tools in parallel. Two hook processes can read the same file, both allow, and the

last writer wins. A 'limit X to 1 per session' rule can be exceeded this way. (claude-code/src/state.ts)

- **[medium] The state file path is hard-coded to /tmp.** On Windows that folder usually does not exist, so saveState throws and every tool call fails with a hook error. The file is also plain JSON anyone on the machine can edit.
- **[medium] OpenClaw id matching is first-in-first-out per tool name.** If two calls of the same tool run concurrently and finish out of order, a failed call can resolve the other call's deferred transition. (openclaw/src/index.ts wireHooks)
- **[low] Session limits are per wrapTools() call.** 'limit sendEmail to 3 per session' resets on every new request unless messages are replayed. A server that forgets to pass messages has no limit at all.

Group 3: quiet behaviours inside the gate

- **[medium] Only the first matching transition fires.** If one net has two enabled transitions listing the same tool, `matching[0]` wins and the other is ignored. A hand-written net that relies on a choice between them will behave unexpectedly. (gate.ts, compose.ts)
- **[medium] autoAdvance refuses ambiguous choices and stops.** That is the safe option, but it is silent. A net with two competing structural transitions never advances and every gated tool stays blocked with no explanation.
- **[medium] Human approval is impossible in two adapters.** claude-code and openclaw pass hasUI: false, so every 'require human-approval' rule is a permanent block there. The docs for those adapters do not say this loudly.
- **[low] Rate limits count attempts, not successes.** The limit template is not deferred, so a push that fails still spends a budget marble.
- **[low] The block reason is a hint for the model.** 'deploy requires a successful call to test first' tells the model exactly which tool to call to unlock deploy. That is the intended recovery path, but it also means a prompt-injected model is guided to the unlock sequence.

Group 4: the engine side

- **[medium] Batch execution can half-apply side effects.** stepBatch uses Promise.all. If one executor throws, the others may already have done their work, but nothing is recorded and the instance is marked failed with the old marking. The docstring asks executors to be idempotent. That is a request, not a guarantee.
- **[medium] Guards are invisible to analysis.** See chapter 10. A workflow can pass --strict and still deadlock at runtime because of a guard.
- **[low] Timeout entries are matched by transition name across a tick.** Expired timeouts add a token even if the transition was re-enabled by a different path since. The scheduler cancels entries when enablement is lost, so this is narrow, but it is subtle.

Group 5: documentation drift found while reading

- README says 'bun dev' runs 'Viewer + API server' and lists Hono as the HTTP router, and CHANGELOG 0.1.0 lists an @petriflow/server package. **There is no server package in packages/.** The viewer proxies /definitions and /workflows to localhost:3000, so its save and load features silently fail.
- CLAUDE.md says petri-ts v0.1.5. Every package.json asks for ^0.2.0.
- CLAUDE.md says 'three runnable Vercel AI SDK agent examples'. README says four. The folder has five (the fifth, 05-openclaw-ops, has no README or test).

- README's quick start says 'bun install, bun test'. Running exactly that on a fresh copy fails every suite with 'Cannot find module @petriflow/gate' because turbo makes test depend on build and the packages resolve to dist/. You must run 'bun run build' first.
- **Windows is not really supported, and nothing says so.** Result of install, build, and test on this machine (Windows 11): **580 pass, 17 fail, 4 errors across 39 files**. Three separate causes: the claude-code adapter writes to /tmp, the cleanup and nuke nets use path.normalize() which turns 'src/' into 'src\' on Windows so path matching tests fail, and the four example test suites load their .rules file with new URL().pathname, which produces '/C:/...' and cannot be opened. The core gate, rules, engine and vercel-ai suites all pass.

12. What needs research

These are open questions a researcher or a serious contributor would want to take on. Some are engineering, some are genuine research.

NEEDS RESEARCH 1. Analyse the composition, not just the parts

Build the product of all active nets (or a synchronised product on shared tool names) and run the analyser on it. That would catch mutually blocking rules and prove 'some tool is always callable' for the whole policy. The state space is the product of small numbers, so it may stay tractable for realistic rule files.

NEEDS RESEARCH 2. Data-aware rules (coloured Petri nets)

'Backup of X before delete of X' needs tokens that carry data. Coloured Petri nets are the standard theory. A DSL extension like 'require backup(path) before delete(path)' could compile to a net plus a generated validateToolCall. Verification of coloured nets is harder; bounded colour domains might be enough here.

NEEDS RESEARCH 3. Guard-aware verification

Guards are small filtrex expressions over a context object. Translating them to a symbolic model checker (an SMT solver, or a bounded model checker) would let analysis include them instead of ignoring them. Then 'no deadlock' would hold for the real system rather than the over-approximation.

NEEDS RESEARCH 4. Tool classification beyond regex

The weakest link is mapping a bash string to a virtual tool. Options to study: parse the shell AST, classify by observed system calls in a sandbox, or use a small model as a classifier with the net as a backstop. Each option has a different false-negative profile, and that profile should be measured, not guessed.

NEEDS RESEARCH 5. Does the model actually recover better?

Version 0.3.0 changed block messages so the LLM can recover. That is a testable hypothesis: measure task success and rule compliance across models with marking-style messages versus constraint-style messages versus no message. Also measure whether the messages help a prompt-injected model find the unlock sequence.

NEEDS RESEARCH 6. Durable, concurrent gate state

The claude-code adapter needs atomic read-modify-write (file locks or a tiny SQLite database like the engine already has). The vercel-ai adapter needs a signed or server-side history. A general 'gate state store' interface, mirroring the engine's WorkflowPersistence, would serve all adapters.

NEEDS RESEARCH 7. Time-based limits

'limit X to 5 per minute' is a natural request. It needs timed Petri nets or a timer token source. The engine already has timeouts; porting a bounded version into SkillNets is open work.

NEEDS RESEARCH 8. Pending-transition hygiene

Define what happens to a deferred transition whose result never arrives: expire after N seconds, cancel on session end, or surface in formatStatus so a human can see it.

NEEDS RESEARCH 9. Fairness of the benchmark

BENCHMARK.md compares against n8n and ReAct built by the same author. An external replication with the n8n workflow reviewed by an n8n expert, and a LangGraph baseline, would strengthen the claim. The footnote already notes the TB-CSPN paper's call-count advantage disappears once batching is equal.

NEEDS RESEARCH 10. A property language for the DSL

Let users write what they expect, for example 'assert delete never before backup' or 'assert deploy at most 2', and have compile() check it against the enumerated states. Today the user must trust the templates. Explicit assertions would turn 'verified' from a state count into a checked claim.

NEEDS RESEARCH 11. Dynamic nets and safety

expandWorkflow() and addNet()/removeNet() change the system while it runs. What properties survive adding a net mid-session? Removing one? There is no analysis of this today.

NEEDS RESEARCH 12. Scale limits of exhaustive enumeration

Rule nets are tiny, but workflow nets grow. At what size does reachableStates() become too slow, and would partial-order reduction or symmetry reduction (standard Petri net techniques) push the limit?

13. Walk the code yourself

13.1 A reading order

Step	Open this	Look for
1	workflows/coffee/definition.ts	A whole net in 70 lines. Match it to the diagram in chapter 2.
2	packages/engine/src/engine.ts	canFireWorkflow and fireWorkflow. Notice how little is here; petri-ts does the rest.
3	packages/gate/src/types.ts	SkillNet and GatedTransition. Every field is documented.
4	packages/gate/src/gate.ts	handleToolCall for a single net. Read it top to bottom with chapter 6 open.
5	packages/gate/src/compose.ts	The four phases. Find the meta snapshot and the TOCTOU re-check.
6	packages/gate/src/advance.ts	autoAdvance and the ambiguity rule.
7	packages/rules/src/compiler.ts	The five templates. Draw each one before reading the next.
8	packages/vercel-ai/src/wrap-tools.ts and replay.ts	How a framework becomes events for the gate.
9	packages/claude-code/src/hook.ts and state.ts	The same idea with a process boundary in the middle.
10	packages/pi-assistant/src/nets/cleanup.ts	What a net looks like when it needs validateToolCall.
11	packages/engine/src/scheduler.ts	The tick loop. Match it to the chapter 9 diagram.
12	workflows/agent-benchmark/agent-benchmark.test.ts	Real property proofs written as tests.

13.2 Commands to try

```
bun install
bun run build                     # required before tests (turbo: test depends on ^build)
bun test                          # all packages
bun test packages/gate            # just the gate
bun packages/cli/src/cli.ts analyse workflows/agent-benchmark/index.ts --strict
bun dev                           # the viewer at http://localhost:5173
```

13.3 Glossary

Word	Meaning in this repo
Place	A cup. A named slot that holds tokens. A circle in diagrams.
Token	A marble. A unit of 'permission' or 'resource' sitting in a place.
Transition	A machine. Consumes tokens from input places, produces tokens in output places.
Marking	The count of tokens in every place. The complete state.
Enabled / canFire	A transition whose input places all have enough tokens.
Reachable states	Every marking you can get to from the start by firing transitions in any order.
Terminal state	A marking where nothing can fire. Good if a terminal place has a token, a deadlock otherwise.

Word	Meaning in this repo
Invariant	A weighted sum of token counts that is the same in every reachable state.
SkillNet	A Petri net whose transitions may list tools, plus freeTools, an optional toolMapper, and optional validators.
Structural transition	type auto with no tools. Fired by autoAdvance, never by the agent.
Deferred	A transition that is allowed on call but only fires on a successful result.
Free tool	A tool a net always allows.
Abstain	A net's verdict when a tool appears nowhere in it.
Jurisdiction	A net has it over a tool if any transition lists that tool.
Virtual tool	The name produced by a toolMapper, such as 'delete' for 'bash rm'.
Guard	A boolean expression (filtrex) that must be true for a workflow transition to fire. Ignored by analysis.
Executor	The async function that performs a workflow transition's side effect and returns a context patch.
Replay	Re-firing transitions from a list of past successful tool calls to rebuild gate state.
Shadow mode	Decide but never block. For trialling rules.

13.4 Self-test

Try to answer, then check against the chapters.

- Why does 'block rm' not need an if-statement? (chapter 3.3)
- A tool called backup returns {ok: false} without throwing. Does delete unlock? What option controls this? (chapters 5.6, 11)
- Two rules: 'require A before B' and 'require B before A'. What happens, and does anything in the repo warn you? (chapter 10)
- In which two adapters can 'require human-approval' never succeed, and why? (chapter 8)
- A workflow passes 'petriflow analyse --strict'. Can it still get stuck at runtime? (chapter 10)
- Why does the vercel-ai gate create a new GateManager inside every wrapTools() call? (chapter 5.3)

End of guide.